5. Networking

Introduction

Today the device leaves its own little "island" for the first time and joins a network. In Zephyr, Wi-Fi is just one of several L2 technologies (Ethernet is another) plugged into the same `net_if` abstraction - your application doesn't talk to the ESP32's Wi-Fi radio directly, it talks to the general networking layer above it. Because joining a network (scanning, associating with an AP, DHCP) takes an unpredictable amount of time and can fail, Zephyr doesn't handle this with a blocking call - instead, it uses a general `net_mgmt` event system: you register a callback, request the connection, and the networking layer notifies you itself once something happens.

A fully commented example is available in zephyr-m5-fit (https://github.com/lenkakt/zephyr-m5-fit) in the `07-wifi` directory - it also has a detailed README explaining all of this step by step.

Graded task assignment

A working Wi-Fi connection **with no password directly in the code**.

"Working" means: on power-up/reset, the device connects to the Wi-Fi network by itself, without the SSID or password appearing anywhere in the source code you submit.

Use that working connection to talk to our simple server's HTTP interface.

The server offers a simple interface:

- `GET /color` returns one word from the set `WHITE`, `RED`, `GREEN`, `BLUE`, `YELLOW`, `PURPLE` - the currently set color.
- `POST /color` with body `change` asks the server to randomly pick a new color.



You'll get the server's address and port in class - it isn't fixed and may differ from one session to the next.

Write an application that:

1. connects to Wi-Fi (see the task above),
2. periodically (e.g. once every few seconds) fetches the current color via `GET /color` and sets the LED strip accordingly,
3. on a button press, sends `POST /color` with body `change`, asking the server for a new random color.

Key parts of the code you'll be working with

A working, commented example for exactly this assignment is available in zephyr-m5-fit (https://github.com/lenkakt/zephyr-m5-fit) in the `08-http` directory - it ties together `07-wifi` (the connection), `02-buttons` (the button), and `03-led-strip`

(the LED strip) with Zephyr's HTTP client (`zephyr/net/http/client.h`). That directory's README explains, among other things:

- why a new TCP connection is opened for every request instead of keeping one persistent connection,
- how the response body is read (`http_client_req()` doesn't return it as a return value, it comes through a callback),
- why reacting to a button press can't make the network call directly in its own callback, and has to hand that work off instead (`k_work_submit()`).

Your own solution doesn't have to look the same - but you should understand **why** `08-http` is built the way it is, not some other way.

What to submit/show at the tutorial

Source code that demonstrably:

- connects to Wi-Fi after a reset with no user intervention,
- displays/sets the current color from `GET /color` on the LED strip,
- sends `POST /color` on a button press and visibly reacts to the color change,
- is sensibly structured (threads, a state machine, etc.).